

Logística

Notas de Clase

Juan Esteban Salamanca

Última modificación: 15 de agosto de 2025

Notas hechas por estudiante de pregrado

Pueden contener errores, omisiones o imprecisiones.

Juan Esteban Salamanca

Índice general

1. JRP - Reaprovisionamiento Conjunto	7
1.1. Reaprovisionamiento sin coordinar	9
1.2. Reaprovisionamiento conjunto $m_j = 1$	12
1.3. Reaprovisionamiento conjunto $m_j \geq 1$	18
2. Coordinación de inventarios	25
2.1. Caso 1-1	25
2.2. Caso 1-n	28
2.3. Ejercicios anidados	30
2.3.1. Caso 1-1-n con énfasis en 1-1	30
2.3.2. Caso 1-1-n con énfasis en 1-n	32
2.3.3. Caso 1-n-1	33
3. Modo de transporte	35
4. El producto en la logística	39
5. Ruta más corta	43
5.1. Algoritmo Dijkstra	44
6. Ruteo	53
6.1. Generalización de heurística Clark and Wright	53
6.2. Macro algoritmo Clark and Wright	56
7. Localización de instalaciones	59
7.1. Modelos basados en distancias	59
7.1.1. Método de la gravedad	59
7.1.2. Método de Weiszfeld	60

8. Diseño de ruta	61
8.1. Algoritmo ADD	61
8.2. Algoritmo DROP	65
8.3. Fórmulas útiles para las macros	68

Capítulo 1

JRP - Reaprovisionamiento Conjunto

Notación reaprovisionamiento

- j : Producto.
- Q_j : Tamaño de la orden del producto j [unds].
- T : Período de reaprovisionamiento conjunto [T].
- O : Costo de ordenar común [\$].
- K_j : Costo fijo de ordenar el producto j [\$].
- h_j : Costo de mantener j [\$/Tiempo].
- i : Tasa de mantenimiento [%/Tiempo].
- c_j : Costo estándar del producto j [\$/unds].
- p_j : Costo por faltante del producto j [\$/unds].
- λ_j : Demanda anual del producto j [unds].
- $n(S_j)$: Valor esperado de faltantes del producto j [unds].
- TC : Costo total [\$].
- $L(Z)$: Función de pérdida estándar.
- m_j : Factor multiplicador del producto j del periodo base T .

Notación reaprovisionamiento**Interpretaciones de NSI:**

1. Relación crítica
2. Probabilidad de no tener faltantes, la probabilidad de tener faltantes estará dada por $1 - \alpha$
3. Probabilidad de tener sobrantes
4. Nivel de servicio tipo I.

Interpretaciones de NSII:

1. Proporción de la demanda que es cubierta con las existencias.

1.1. Reaprovisionamiento sin coordinar

Ecuaciones reaprovisionamiento sin coordinar

La que usabamos en control: Importante para no depender de T

$$T = Q/\lambda$$

Caso determinístico:

★ **Parámetros:**

SS_j = Nos lo tienen que dar

$$Q_j = EOQ_j = \sqrt{\frac{2(K_j + O)\lambda_j}{h_j}} = \sqrt{\frac{2(K_j + O)\lambda_j}{i_j c_j}}$$

$$\bar{I}_j = Q_j/2 + SS_j$$

★ **Costos:**

$$TC = C.Oordenar + C.Mantener$$

$$TC = \sum_j \frac{\lambda_j}{Q_j} (O + K_j) + \sum_j \bar{I}_j * h_j$$

$$= \sum_j \frac{\lambda_j}{Q_j} (O + K_j) + \sum_j \left(\frac{Q_j}{2} + SS_j \right) * i * c_j$$

Caso estocástico:

★ **Parámetros (ajustado a una distr. normal):**

$$Q_j = EOQ_j = \sqrt{\frac{2(K_j + O)\lambda_j}{h_j}} = \sqrt{\frac{2(K_j + O)\lambda_j}{i_j c_j}}$$

$$S = \mu_{T+\tau} + Z\sigma_{T+\tau} = \mu_{T+\tau} + SS$$

$$SS = Z * \sigma_{T+\tau}$$

Nota: SS está indexado porque depende de la desviación estandar de cada demanda dada su fórmula:

$$Z = \frac{S - \mu_{T+\tau}}{\sigma_{T+\tau}} = \text{DISTR.NORM.INV}(\alpha; 0; 1)$$

$$L(Z) = \text{DISTR.NORM}(z; 0; 1; 0) - z * (1 - \text{DISTR.NORM.ESTAND}(z))$$

$$n(S) = \sigma_{T+\tau} * L(Z)$$

$$NSI = \alpha = \mathcal{P}(x_{T+\tau} \leq S) = F(Q^*)$$

$$NSII = 1 - \frac{n(S)}{\bar{Q}} = 1 - \frac{\sigma_{T+\tau} L(Z)}{\bar{Q}}$$

Ecuaciones reaprovisionamiento sin coordinar

★ **Costos:**

$$TC = C.Ordenar + C.Mantener + C.Faltantes$$

$$C_{Ordenar} = \sum_j \frac{\lambda_j}{Q_j} (O + K_j)$$

$$C_{Mantener} = \sum_j \bar{I}_j * h_j = \sum_j \left(\frac{Q_j}{2} + SS_j \right) ic_j$$

$$C_{Faltantes} = \sum_j \frac{\lambda_j}{Q_j} (p_j n(S_j))$$

$$\begin{aligned} TC &= \sum_j \frac{\lambda_j}{Q_j} (O + K_j) + \sum_j \bar{I}_j * h_j + \sum_j \frac{\lambda_j}{Q_j} (p_j n(S_j)) \\ &= \sum_j \frac{\lambda_j}{Q_j} (O + K_j) + \sum_j \left(\frac{Q_j}{2} + SS_j \right) ic_j + \sum_j \frac{\lambda_j}{Q_j} (p_j n(S_j)) \end{aligned}$$

Receta: Diseñar política Up-To-Level (ST) para una serie de productos dado un escenario determinístico.

1. Consideramos matriz inicial que nos da el enunciado, deberían darnos todos los costos y demandas. No necesitaremos la desviación estándar porque es determinístico.
2. Chequear que todo esté en la misma temporalidad, si no lo está, ponerlo en la misma temporalidad.
3. Encontrar matriz de parámetros que necesitaremos para los costos más adelante.

Producto	Producto 1	Producto 2	...
Q	EOQ 1.1	EOQ 1.1	...
SS	Nos dicen	Nos dicen	...
$\bar{I}$	$Q_1/2 + SS_1$	$Q_2/2 + SS_2$	...

$$EOQ = \sqrt{\frac{2K_j\lambda_j}{i_j c_j}} \quad (1.1)$$

4. Encontrar costos

$$TC = C.Ordenar + C.Mantener$$

$$C.Ordenar = \sum_j \frac{\lambda_j}{Q_j} (O + K_j)$$

$$C.Mantener = \sum_j \bar{I}_j * h_j = \sum_j \left(\frac{Q_j}{2} + SS_j \right) * i * c_j$$

$$\begin{aligned} TC &= \sum_j \frac{\lambda_j}{Q_j} (O + K_j) + \sum_j \bar{I}_j * h_j \\ &= \sum_j \frac{\lambda_j}{Q_j} (O + K_j) + \sum_j \left(\frac{Q_j}{2} + SS_j \right) * i * c_j \end{aligned}$$

Notemos que si queremos discriminar costos por producto, en vez de la sumatoria ponemos un $\forall j$ para cada costo que vayamos a calcular.

1.2. Reaprovisionamiento conjunto $m_j = 1$

Ecuaciones reaprovisionamiento conjunto $m_j = 1$ bajo política S-T

La que usábamos en control: Importante para inventarios promedio

$$Q = \lambda * T$$

Caso determinístico:

★ **Parámetros:**

Asumiendo que el stock de seguridad es una fracción de Q tal que

$SS_j = \beta_j * Q_j = \beta_j * \lambda_j * T$, $\beta_j \in [0, 1)$ entonces

$$T^* = \sqrt{\frac{O + \sum_j K_j}{\sum_j \left(\left(\frac{1}{2} + \beta_j \right) \lambda_j i_j c_j \right)}}$$

★ **Costos:**

$$TC = C.Oordenar + C.Mantener$$

$$\begin{aligned} TC &= \frac{1}{T^*} \left(O + \sum_j K_j \right) + \sum_j \left(\frac{Q_j}{2} + SS_j \right) i_j c_j \\ &= \frac{1}{T^*} \left(O + \sum_j K_j \right) + \sum_j \bar{I} * i_j * c_j \end{aligned}$$

Caso estocástico (ajustamos todo a una distr. normal):

★ **Parámetros:**

$$S = \mu_{T+\tau} + Z\sigma_{T+\tau} = \mu_{T+\tau} + SS$$

$$SS = Z * \sigma_{T+\tau}$$

Nota: SS está indexado porque depende de la desviación estandar de cada demanda dada su fórmula:

$$Z = \frac{S - \mu_{T+\tau}}{\sigma_{T+\tau}} = \text{DISTR.NORM.INV}(\alpha; 0; 1)$$

$$L(Z) = \text{DISTR.NORM}(z; 0; 1; 0) - z * (1 - \text{DISTR.NORM.ESTAND}(z))$$

$$n(S) = \sigma_{T+\tau} * L(Z)$$

$$NSI = \alpha = \mathcal{P}(x_{T+\tau} \leq S) = F(Q^*)$$

$$NSII = 1 - \frac{n(S)}{\bar{Q}} = 1 - \frac{\sigma_{T+\tau} L(Z)}{\bar{Q}}$$

Ecuaciones reaprovisionamiento conjunto $m_j = 1$ bajo política S-T★ **Costos:**

$$TC = C.Oordenar + C.Mantener + C.Faltantes$$

$$C_{Ordenar} = \frac{1}{T^*} \left(O + \sum_j K_j \right)$$

$$C_{Mantener} = \sum_j \bar{I}_j * h_j = \sum_j \left(\frac{Q_j}{2} + SS_j \right) ic_j = \sum_j \left(\frac{\lambda_j * T^*}{2} + SS_j \right) ic_j$$

$$C_{Faltantes} = \sum_j \frac{p_j n(S_j)}{T}$$

$$TC = \frac{1}{T} \left(O + \sum_j K_j \right) + \sum_j \left(\frac{Q_j}{2} + SS_j \right) ic_j + \sum_j \frac{p_j n(S_j)}{T}$$

Receta: Diseñar política Up-To-Level (ST) para una serie de productos sujeto a un NSI en específico (caso estocástico).

La lógica es llegar a los costos (anuales en general) de la política, y para esto, debemos encontrar todos los parámetros de la función de costos totales promedio.

1. Considerar matriz de costos, tasas, desviaciones, etc. que nos dan inicialmente.
2. Chequear que todo esté en la misma temporalidad, si no lo está, ponerlo en la misma temporalidad.
3. Sacamos un solo T^* dado que todos los $m_j = 1$:

$$T^* = \sqrt{\frac{2 \left(O + \sum_j K_j \right)}{i \sum_j c_j \lambda_j}}$$

4. Pasar T^* a la unidad de tiempo más chiquita (generalmente días) y redondear a entero más cercano. Así encontraremos el valor de T .

5. Sacar matriz de parámetros que usaremos para encontrar costos totales:

	Producto 1	Producto 2	...
$T + \tau$	$T + \tau_1$	$T + \tau_2$	...
$\mu(T + \tau)$	$\lambda_1 \cdot (T + \tau_1)$	$\lambda_2 \cdot (T + \tau_2)$	...
$\sigma(T + \tau)$	$\sigma_1 \cdot \sqrt{T + \tau_1}$	$\sigma_2 \cdot \sqrt{T + \tau_2}$	...
Z	Eq 1.2	Eq 1.2	...
$L(Z)$	Eq 1.3	Eq 1.3	...
$S^!$	$\mu_{T+\tau} + \sigma_{T+\tau} \cdot Z$	$\mu_{T+\tau} + \sigma_{T+\tau} \cdot Z$	...
$SS^!$	$\sigma_{T+\tau} \cdot Z$	$\sigma_{T+\tau} \cdot Z$	...
Q	$\lambda_1 \cdot T$	$\lambda_2 \cdot T$	...
$\bar{I}$	$Q_1/2 + SS_1$	$Q_2/2 + SS_2$	...
$n(S)^!$	$\sigma_{T+\tau} \cdot L(Z)$	$\sigma_{T+\tau} \cdot L(Z)$	...

$$Z = \frac{S - \mu_{T+\tau}}{\sigma_{T+\tau}} = \text{DISTR.NORM.INV}(\alpha; 0; 1) \quad (1.2)$$

$$L(Z) = \text{DISTR.NORM}(z; 0; 1; 0) - z * (1 - \text{DISTR.NORM.ESTAND}(z)) \quad (1.3)$$

6. Con esto ya deberíamos tener todo lo necesario para encontrar los costos totales, así pues, computaremos:

$$C_{\text{Ordenar}} = \frac{1}{T} \left(O + \sum_j K_j \right)$$

$$C_{\text{Mantener}} = \sum_j (\bar{I} + SS_j) ic_j = \sum_j \left(\frac{Q_j}{2} + SS_j \right) ic_j = \sum_j \left(\frac{\lambda_j * T}{2} + SS_j \right) ic_j$$

$$C_{\text{Faltantes}} = \sum_j \frac{p_j n(S_j)}{T}$$

$$TC = \frac{1}{T} \left(O + \sum_j K_j \right) + \sum_j \left(\frac{Q_j}{2} + SS_j \right) ic_j + \sum_j \frac{p_j n(S_j)}{T}$$

Notemos que si queremos discriminar costos por producto, en vez de la sumatoria ponemos un $\forall j$ para cada costo que vayamos a calcular.

Nota:

El costo de ordenar común O solo se toma una vez. No es uno por cada producto porque como $m_j = 1 \forall j$, todo se pide al tiempo.

[!]Esto debe ir indexado para todo trabajo j , no lo puse porque implicaría meterle subíndices a los subíndices y queda feito :/.

Receta: Diseñar política Up-To-Level (ST) para una serie de productos (caso determinístico).

1. Considerar matriz de costos, tasas, etc. que nos dan inicialmente.
2. Chequear que todo esté en la misma temporalidad, si no lo está, ponerlo en la misma temporalidad.
3. Sacamos un solo T dado que todos los $m_j = 1$. Para esto, supondremos que el stock de seguridad está dado de la forma $SS = \beta_j * Q_j$:

$$\begin{aligned}
 TC &= \frac{1}{T} \left(O + \sum_j K_j \right) + \sum_j \left(\left(\frac{1}{2} + \beta_j \right) \lambda_j T \right) ic_j \\
 \frac{dTC}{dT} &= -\frac{1}{T^2} \left(O + \sum_j K_j \right) + \sum_j \left(\left(\frac{1}{2} + \beta_j \right) \lambda_j ic_j \right) = 0 \\
 \Rightarrow \sum_j \left(\left(\frac{1}{2} + \beta_j \right) \lambda_j ic_j \right) &= \frac{1}{T^2} \left(O + \sum_j K_j \right) \\
 \Rightarrow T^2 &= \frac{O + \sum_j K_j}{\sum_j \left(\left(\frac{1}{2} + \beta_j \right) \lambda_j ic_j \right)} \\
 \Rightarrow T^* &= \sqrt{\frac{O + \sum_j K_j}{\sum_j \left(\left(\frac{1}{2} + \beta_j \right) \lambda_j ic_j \right)}}
 \end{aligned}$$

$$T^* = \sqrt{\frac{O + \sum_j K_j}{\sum_j \left(\left(\frac{1}{2} + \beta_j \right) \lambda_j ic_j \right)}}$$

4. Pasar T^* a la unidad de tiempo más chiquita (generalmente días) y redondear a entero más cercano. El resultado será un nuevo tiempo T que no es óptimo, pero será usado de ahora en adelante.
5. Sacar matriz de parámetros que usaremos para encontrar costos totales:

Producto	Producto 1	Producto 2	...
Q	$\lambda_1 * T$	$\lambda_2 * T$	...
SS	Nos dicen	Nos dicen	...
$\bar{I}$	$Q_1/2 + SS_1$	$Q_2/2 + SS_2$	...

6. Sacar costos con parámetros encontrados:

$$TC = C.OordenarIndividual + C.OordenarConjunto + C.Mantener$$

$$C.OordenarComún = \frac{O}{T}$$

$$C.OrdenarIndividual = \frac{\sum_j K_j}{T}$$

$$C.Mantener = \sum_j \bar{I}_j h_j = \sum_j \left(\frac{Q_j}{2} + SS_j \right) * i * c_j = \sum_j \left(\frac{Q_j}{2} + \beta_j Q_j \right) * i * c_j$$

$$\begin{aligned} TC &= \frac{1}{T} \left(O + \sum_j K_j \right) + \sum_j \bar{I}_j h_j \\ &= \frac{1}{T} \left(O + \sum_j K_j \right) + \sum_j \left(\frac{Q_j}{2} + SS_j \right) * i * c_j \\ &= \frac{1}{T} \left(O + \sum_j K_j \right) + \sum_j \left(\frac{Q_j}{2} + \beta_j Q_j \right) * i * c_j \end{aligned}$$

1.3. Reaprovisionamiento conjunto $m_j \geq 1$

Ecuaciones reaprovisionamiento $m_j \geq 1$

Caso determinístico

★ Parámetros:

$$\text{Producto base} = \arg \min_j \left(\frac{K_j}{c_j \lambda_j} \right)$$

$$m_{j^*} = 1$$

$$m_j = \max \left\{ \text{redondear} \left(\sqrt{\frac{K_j}{\lambda_j c_j} \cdot \frac{\lambda_j^* c_j^*}{O + K_j^*}} \right), 1 \right\}$$

Asumiendo que el stock de seguridad es una fracción de Q tal que $SS_j = \beta_j * Q_j = \beta_j * \lambda_j * T * m_j$, $\beta_j \in [0,1)$ entonces

$$T^*(\vec{m}) = \sqrt{\frac{[O + \sum (\frac{K_j}{m_j})]}{\sum h_j \lambda_j m_j * (1/2 + \beta_j)}} = \sqrt{\frac{[O + \sum (\frac{K_j}{m_j})]}{\sum i_j c_j \lambda_j m_j * (1/2 + \beta_j)}}$$

$$T_j = m_j T$$

$$SS = \text{Nos lo dan}$$

$$Q_j^* = T_j * \lambda_j$$

$$\bar{I} = \frac{Q_j^*}{2} + SS_j$$

★ Costos:

$$CT = C.\text{OrdenarComun} + C.\text{OrdenarIndividual} + C.\text{Mantener}$$

$$C.\text{OrdenarComun}_j = \frac{O}{T}$$

$$C.\text{OrdenarIndividual}_j = \frac{K_j}{T_j}$$

$$C.\text{Mantener} = \bar{I}_j * h_j = \left(\frac{Q_j^*}{2} + SS_j \right) * i_j * c_j$$

$$\begin{aligned} CT &= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{Q_j^*}{2} + SS_j \right) * i_j * c_j \\ &= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{\lambda_j * T_j}{2} + SS_j \right) * i_j * c_j \\ &= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{\lambda_j * T^* * m_j}{2} + SS_j \right) * i_j * c_j \end{aligned}$$

Ecuaciones reaprovisionamiento $m_j \geq 1$ **Caso estocástico:****★ Parámetros:**

$$\text{Producto base} = \arg \min_j \left(\frac{K_j}{c_j \lambda_j} \right)$$

$$m_{j^*} = 1$$

$$m_j = \max \left\{ \text{redondear} \left(\sqrt{\frac{K_j}{\lambda_j c_j} \cdot \frac{\lambda_j^* c_j^*}{O + K_j^*}} \right), 1 \right\}$$

$$T^*(\vec{m}) = \sqrt{\frac{2 \left[O + \sum \left(\frac{K_j}{m_j} \right) \right]}{\sum h_j \lambda_j m_j}} = \sqrt{\frac{2 \left[O + \sum \left(\frac{K_j}{m_j} \right) \right]}{\sum i_j c_j \lambda_j m_j}}$$

$$T_j = m_j T^*$$

$$\mu_{T+\tau} = \lambda * (T + \tau)$$

$$\sigma_{T+\tau} = \sigma * \sqrt{T + \tau}$$

$$Z = \frac{S - \mu_{T+\tau}}{\sigma_{T+\tau}} = \text{DISTR.NORM.INV}(\alpha; 0; 1)$$

$$L(Z) = \text{DISTR.NORM}(z; 0; 1; 0) - z * (1 - \text{DISTR.NORM.ESTAND}(z))$$

$$S = \mu_{T+\tau} + Z \sigma_{T+\tau} = \mu_{T+\tau} + SS$$

$$SS = Z * \sigma_{T+\tau}$$

$$Q_j^* = T_j * \lambda_j$$

$$\bar{I}_j = \frac{Q_j^*}{2} + SS_j$$

$$n(S) = \sigma_{T+\tau} * L(Z)$$

$$NSI = \alpha = \mathcal{P}(x_{T+\tau} \leq S) = F(Q^*)$$

$$NSII = 1 - \frac{n(S)}{\bar{Q}} = 1 - \frac{\sigma_{T+\tau} L(Z)}{\bar{Q}}$$

★ Costos:

$$CT = C.\text{OrdenarComun} + C.\text{OrdenarIndividual} + C.\text{Mantener} + C.\text{Faltantes}$$

$$C.\text{OrdenarComun}_j = \frac{O}{T}$$

$$C.\text{OrdenarIndividual}_j = \frac{K_j}{T_j}$$

$$C.\text{Mantener} = \bar{I}_j * h_j = \left(\frac{Q_j^*}{2} + SS_j \right) * i_j * c_j$$

$$C.\text{Faltantes} = \sum_j \frac{1}{T_j} (p_j n(S_j))$$

Ecuaciones reaprovisionamiento $m_j \geq 1$

$$\begin{aligned}
CT &= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{Q_j^*}{2} + SS_j \right) * i_j * c_j + \sum_j \frac{1}{T_j} (p_j n(S_j)) \\
&= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{\lambda_j * T_j}{2} + SS_j \right) * i_j * c_j + \sum_j \frac{1}{T_j} (p_j n(S_j)) \\
&= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{\lambda_j * T * m_j}{2} + SS_j \right) * i_j * c_j + \sum_j \frac{1}{T_j} (p_j n(S_j))
\end{aligned}$$

Receta: Diseñar política Up-To-Level (ST) para una serie de productos sujeto a escenario determinístico.

1. Considerar matriz de costos, tasas, desviaciones, etc. que nos dan inicialmente.
2. Chequear que todo esté en la misma temporalidad, si no lo está, ponerlo en la misma temporalidad.
3. Sacar matriz de parámetros que usaremos para encontrar costos totales:

Productos	Producto 1	Producto 2	...
Orden	$\frac{K_j}{c_j \lambda_j}$	$\frac{K_j}{c_j \lambda_j}$	...
Paso intermedio: sacar $\frac{\lambda_j^* c_j^*}{O + K_j^*}$			
m_j fórmula	Aplicar 1.4	Aplicar 1.4	...
m_j seleccionado	Poner $m_j^* = 1$	Poner $m_j^* = 1$	...
m_j redondeado	$\max\{1, \text{redondear}(m_j)\}$	$\max\{1, \text{redondear}(m_j)\}$	...
$\frac{K_j}{m_j}$	$\frac{K_j}{m_j}$	$\frac{K_j}{m_j}$	...
$m_j c_j i_j \lambda_j$	$m_j c_j i_j \lambda_j$	$m_j c_j i_j \lambda_j$	...
Paso intermedio: sacar T^* con 1.5			
Paso intermedio: Sacar T aproximando (Ver Aclaración)			
T_j	$m_j(\text{red}) * T$	$m_j(\text{red}) * T$	...
SS	Nos lo dan	Nos lo dan	...
Q^*	$\lambda_j * T_j$	$\lambda_j * T_j$	...
$\bar{I}$ prom	$Q_j^* / 2 + SS_j$	$Q_j^* / 2 + SS_j$	...

$$m_j = \sqrt{\frac{K_j}{\lambda_j c_j} \cdot \frac{\lambda_j^* c_j^*}{O + K_j^*}} \quad (1.4)$$

$$T^*(\vec{m}) = \sqrt{\frac{\left[O + \sum \left(\frac{K_j}{m_j}\right)\right]}{\sum h_j \lambda_j m_j * (1/2 + \beta_j)}} = \sqrt{\frac{\left[O + \sum \left(\frac{K_j}{m_j}\right)\right]}{\sum i_j c_j \lambda_j m_j * (1/2 + \beta_j)}} \quad (1.5)$$

Aclaración: Cuando decimos “aproximar” nos referimos a pasar el valor de T^* a la unidad de tiempo más pequeña, redondear al entero más cercano y volver a la unidad de tiempo que nos pide el enunciado usando el valor redondeado. Este nuevo valor será T , no es óptimo, pero será usado de ahora en adelante.

4. Sacamos costos

$$CT = C.OordenarComun + C.OordenarIndividual + C.Mantener$$

$$C.OordenarComun_j = \frac{O}{T}$$

$$C.OordenarIndividual_j = \frac{K_j}{T_j}$$

$$C.Mantener = \bar{I}_j * h_j = \left(\frac{Q_j^*}{2} + SS_j \right) * i_j * c_j$$

$$\begin{aligned} CT &= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{Q_j^*}{2} + SS_j \right) * i_j * c_j \\ &= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{\lambda_j * T_j}{2} + SS_j \right) * i_j * c_j \\ &= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{\lambda_j * T * m_j}{2} + SS_j \right) * i_j * c_j \end{aligned}$$

Receta: Diseñar política Up-To-Level (ST) para una serie de productos sujeto a un NSI en específico (caso estocástico).

1. Considerar matriz de costos, tasas, desviaciones, etc. que nos dan inicialmente.
2. Chequear que todo esté en la misma temporalidad, si no lo está, ponerlo en la misma temporalidad.
3. Sacar matriz de heurística:

Productos	Producto 1	Producto 2	...
Orden	$\frac{K_j}{c_j \lambda_j}$	$\frac{K_j}{c_j \lambda_j}$	...
Paso intermedio: sacar $\frac{\lambda_j^* c_j^*}{O + K_j^*}$			
m_j fórmula	Aplicar 1.6	Aplicar 1.6	...
m_j seleccionado	Poner $m_j^* = 1$	Poner $m_j^* = 1$	...
m_j redondeado	$\max\{1, \text{redondear}(m_j)\}$	$\max\{1, \text{redondear}(m_j)\}$	...
$\frac{K_j}{m_j}$	$\frac{K_j}{m_j}$	$\frac{K_j}{m_j}$	...
$m_j c_j i_j \lambda_j$	$m_j c_j i_j \lambda_j$	$m_j c_j i_j \lambda_j$	...
Paso intermedio: sacar T^* con 1.7 y aproximar			
Paso intermedio: Sacar T aproximando (Ver Aclaración)			
T_j	$(m_j \text{ redondeado}) * T$	$(m_j \text{ redondeado}) * T$	...

$$m_j = \sqrt{\frac{K_j}{\lambda_j c_j} \cdot \frac{\lambda_j^* c_j^*}{O + K_j^*}} \quad (1.6)$$

$$T^*(\vec{m}) = \sqrt{\frac{2 \left[O + \sum \left(\frac{K_j}{m_j} \right) \right]}{\sum h_j \lambda_j m_j}} = \sqrt{\frac{2 \left[O + \sum \left(\frac{K_j}{m_j} \right) \right]}{\sum i_j c_j \lambda_j m_j}} \quad (1.7)$$

Aclaración: Cuando decimos “aproximar” nos referimos a pasar el valor de T^* a la unidad de tiempo más pequeña, redondear al entero más cercano y volver a la unidad de tiempo que nos pide el enunciado usando el valor redondeado. Este nuevo valor será T , no es óptimo, pero será usado de ahora en adelante.

4. Sacar matriz de parámetros de distribución:

	Producto 1	Producto 2	...
$T_j + \tau$	$T_j + \tau_1$	$T_j + \tau_2$	...
$\mu(T_j + \tau)$	$\lambda_1 \cdot (T_j + \tau_1)$	$\lambda_2 \cdot (T_j + \tau_2)$	...
$\sigma(T_j + \tau)$	$\sigma_1 \cdot \sqrt{T_j + \tau_1}$	$\sigma_2 \cdot \sqrt{T_j + \tau_2}$	...
Z	Eq 1.8	Eq 1.8	...
$L(Z)$	Eq 1.9	Eq 1.9	...
$S^!$	$\mu_{T_j+\tau} + \sigma_{T_j+\tau} \cdot Z$	$\mu_{T_j+\tau} + \sigma_{T_j+\tau} \cdot Z$	...
$SS^!$	$\sigma_{T_j+\tau} \cdot Z$	$\sigma_{T_j+\tau} \cdot Z$	...
Q	$\lambda_j \cdot T_j$	$\lambda_j \cdot T_j$	...
$\bar{I}$	$Q_j/2 + SS_j$	$Q_j/2 + SS_j$	...
$n(S)^!$	$\sigma_{T+\tau} \cdot L(Z)$	$\sigma_{T+\tau} \cdot L(Z)$	...

$$Z = \frac{S - \mu_{T+\tau}}{\sigma_{T+\tau}} = \text{DISTR.NORM.INV}(\alpha; 0; 1) \quad (1.8)$$

[!]Esto debe ir indexado para todo trabajo j , no lo puse porque implicaría meterle subíndices a los subíndices y queda feito :/.

$$L(Z) = \text{DISTR.NORM}(z; 0; 1; 0) - z * (1 - \text{DISTR.NORM.ESTAND}(z)) \quad (1.9)$$

5. Sacar costos

$$CT = C.\text{OrdenarComun} + C.\text{OrdenarIndividual} + C.\text{Mantener} + C.\text{Faltantes}$$

$$C.\text{OrdenarComun}_j = \frac{O}{T}$$

$$C.\text{OrdenarIndividual}_j = \frac{K_j}{T_j}$$

$$C.\text{Mantener} = \bar{I}_j * h_j = \left(\frac{Q_j^*}{2} + SS_j \right) * i_j * c_j$$

$$C.\text{Faltantes} = \frac{1}{T_j} (p_j n(S_j))$$

$$\begin{aligned} CT &= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{Q_j^*}{2} + SS_j \right) * i_j * c_j + \sum_j \frac{1}{T_j} (p_j n(S_j)) \\ &= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{\lambda_j * T_j}{2} + SS_j \right) * i_j * c_j + \sum_j \frac{1}{T_j} (p_j n(S_j)) \\ &= \frac{O}{T} + \sum_j \frac{K_j}{T_j} + \sum_j \left(\frac{\lambda_j * T * m_j}{2} + SS_j \right) * i_j * c_j + \sum_j \frac{1}{T_j} (p_j n(S_j)) \end{aligned}$$

Capítulo 2

Coordinación de inventarios

Notación Coordinación de inventarios

- λ : Demanda del minorista [unidades/Tiempo]
- K_W : Costo de pedido del centro de distribución [\$]
- K_R : Costo de pedido del minorista [\$]
- C_W : Costo unitario del ítem en el CD [\$/unidad]
- C_R : Costo unitario del ítem en el minorista [\$/unidad]
- i : Tasa anual de manejo de inventarios [\$/Tiempo]
- C'_W : Costo del ítem en el CD [\$/unidad]
- C'_R : Costo agregado o incremental por ítem en el minorista [\$/unidad]
- λ_0 : Tasa de consumo asociada a la bodega [unds/Tiempo]
- T_0 : Tiempo de ciclo de consumo asociado a la bodega [Tiempo].
- K_0, h_0 : Costos asociados a la bodega [\$]

2.1. Caso 1-1

En este tipo de ejercicios, tenemos una bodega y un minorista



Ecuaciones Coordinación de inventarios caso 1-1

La de control

$$Q_R = \lambda * T_R$$

$$Q_W = \lambda * T_W = \lambda * n * T_R$$

Relación entre bodega (Warehouse) y minoristas (Retailers)

$$T_W = nT_R$$

$$Q_W = nQ_R$$

Inventario promedio total

$$\frac{Q_W}{2} + SS_W$$

Inventario promedio minorista

$$\frac{Q_R}{2} + SS_R$$

Costos:**★ Costo total de mantener**

$$\begin{aligned} & iC'_W \left(\frac{Q_W}{2} + SS_W \right) + iC'_R \left(\frac{Q_R}{2} + SS_R \right) \\ &= iC_W \left(\frac{Q_W}{2} + SS_W \right) + i(C_R - C_W) \left(\frac{Q_R}{2} + SS_R \right) \\ &= iC_W \left(\frac{Q_W}{2} + SS_W \right) + i(C_R - C_W) \left(\frac{nQ_W}{2} + SS_R \right) \end{aligned}$$

★ Costo total de realizar pedido

$$\begin{aligned} \frac{K_W \lambda}{Q_W} + \frac{K_R \lambda}{Q_R} &= \frac{K_W \lambda}{nQ_R} + \frac{K_R \lambda}{Q_R} \\ &= \frac{\lambda}{Q_R} (K_R + K_W) \end{aligned}$$

Ecuaciones Coordinación de inventarios caso 1-1

★ Costo total

C.Realizar + C.Mantener

$$\begin{aligned}
&= \frac{K_W \lambda}{Q_W} + i_W C'_W \left(\frac{Q_W}{2} + SS_W \right) + \frac{K_R \lambda}{Q_R} + i_R C'_R \left(\frac{Q_R}{2} + SS_R \right) \\
&= \frac{K_W \lambda}{Q_W} + i_W C_W \left(\frac{Q_W}{2} + SS_W \right) + \frac{K_R \lambda}{Q_R} + i_R (C_R - C_W) \left(\frac{Q_R}{2} + SS_R \right) \\
&= \frac{K_W \lambda}{n Q_R} + i_W C_W \left(\frac{n Q_R}{2} + SS_W \right) + \frac{K_R \lambda}{Q_R} + i_R (C_R - C_W) \left(\frac{Q_R}{2} + SS_R \right)
\end{aligned}$$

Receta (caso determinístico): Suponiendo que nos dan info. de costos, tasa de demanda y tasa de retención de inventario.

1. Poner todo en una misma temporalidad (usar las conversiones que nos dan).
2. Reemplazar en la ecuación de costos $Q_W = n Q_R$
3. Identificar stocks de seguridad. Asumiremos que los stocks de seguridad son combinaciones lineales de Q_R , Q_W y también de la tasa de producción λ :

$$SS_W = \alpha Q_R + \beta Q_W + \theta \lambda = Q_R(\alpha + \beta n) + \theta \lambda$$

$$SS_R = \gamma Q_R + \sigma Q_W + \delta \lambda = Q_R(\gamma + \sigma n) + \delta \lambda$$

$$\alpha, \beta, \gamma, \sigma, \theta, \delta \in [0, 1)$$

Por ende, la función de costos totales quedará tal que:

$$\begin{aligned}
CT(Q_R) &= \frac{K_W \lambda}{Q_W} + i_W C_W \left(\frac{Q_W}{2} + SS_W \right) + \frac{K_R \lambda}{Q_R} + i_R (C_R - C_W) \left(\frac{Q_R}{2} + SS_R \right) \\
&= \frac{K_W \lambda}{n Q_R} + i_W C_W \left(\frac{n Q_R}{2} + Q_R(\alpha + \beta n) + \theta \lambda \right) \\
&\quad + \frac{K_R \lambda}{Q_R} + i_R (C_R - C_W) \left(\frac{Q_R}{2} + Q_R(\gamma + \sigma n) + \delta \lambda \right) \\
&= \frac{\lambda}{Q_R} \left(\frac{K_W}{n} + K_R \right) + Q_R \left[i_W C_W \left(\frac{n}{2} + \alpha + \beta n \right) + i_R (C_R - C_W) \left(\frac{1}{2} + \gamma + \sigma n \right) \right] \\
&\quad + \lambda [i_W C_W \theta + i_R (C_R - C_W) \delta]
\end{aligned}$$

4. Derivar función de costos totales con respecto a Q_R y despejar Q_R :

$$\begin{aligned}
CT(Q_R) &= \frac{\lambda}{Q_R} \left(\frac{K_W}{n} + K_R \right) + Q_R \left[i_W C_W \left(\frac{n}{2} + \alpha + \beta n \right) + i_R (C_R - C_W) \left(\frac{1}{2} + \gamma + \sigma n \right) \right] \\
&\quad + \lambda [i_W C_W \theta + i_R (C_R - C_W) \delta]
\end{aligned}$$

$$CT'(Q_R) = -\frac{\lambda \left(\frac{K_W}{n} + K_R \right)}{Q_R^2} + i_W C_W \left(\frac{n}{2} + \alpha + \beta n \right) + i_R (C_R - C_W) \left(\frac{1}{2} + \gamma + \sigma n \right)$$

El término constante con λ no afecta la derivada.

$$Q_R^* = \sqrt{\frac{\lambda \left(\frac{K_W}{n} + K_R \right)}{i_W C_W \left(\frac{n}{2} + \alpha + \beta n \right) + i_R (C_R - C_W) \left(\frac{1}{2} + \gamma + \sigma n \right)}}$$

5. Dejar expresados T_R, Q_W en función de n^* y Q_R :

$$Q_R = \lambda T_R \Rightarrow T_R = \frac{Q_R}{\lambda}, \quad Q_W = n Q_R$$

6. Inicializar valor de n^* .

7. Expresar función de costos con los parámetros que dejamos expresados.

8. Solver para minimizar función de costos tal que $n^* \in \mathbb{N}$ y $n^* \geq 1$

$$CT(Q_R) = \frac{\lambda}{Q_R} \left(\frac{K_W}{n} + K_R \right) + Q_R \left[i_W C_W \left(\frac{n}{2} + \alpha + \beta n \right) + i_R (C_R - C_W) \left(\frac{1}{2} + \gamma + \sigma n \right) \right] + \lambda [i_W C_W \theta + i_R (C_R - C_W) \delta]$$

Costos individuales:

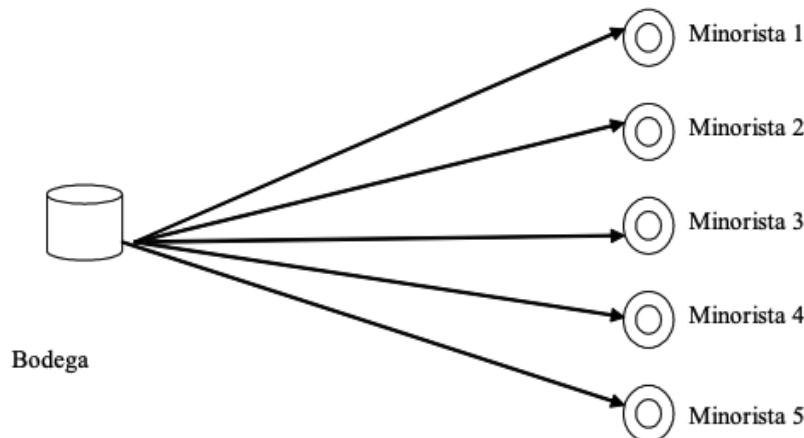
$$C.Adq_W = \frac{\lambda K_W}{Q_W}, \quad C.Adq_R = \frac{\lambda K_R}{Q_R}$$

$$C.Mant_W = i_W C_W \left(\frac{Q_W}{2} + \alpha Q_R + \beta Q_W + \theta \lambda \right)$$

$$C.Mant_R = i_R (C_R - C_W) \left(\frac{Q_R}{2} + \gamma Q_R + \sigma Q_W + \delta \lambda \right)$$

2.2. Caso 1-n

En este caso tendremos una bodega y $n > 1$ minoristas.



Ecuaciones Coordinación de inventarios caso 1-n

Heurística:

$$T_j^* = \sqrt{\frac{2K_j}{h_j\lambda_j}}$$

$$T_0 = \begin{cases} T_0^*, & \text{si } J = \emptyset \\ \sqrt{\frac{2(K_0 + \sum_{j \in J} K_j)}{\lambda_0 h_0 + \sum_{j \in J} \lambda_j h_j}}, & \text{si } J \neq \emptyset \end{cases}$$

$$\begin{aligned} \min CT_j &= \frac{n_j K_j}{T_0} + h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right) \\ &= \frac{K_j}{T_j} + h_j \left(\frac{\lambda_j T_j}{2} + SS_j \right) \\ &\quad \text{Si } SS_j \text{ es una fracción } \beta_j \text{ de } \lambda_j: \\ &= \frac{K_j}{T_j} + h_j \lambda_j \left(\frac{T_j}{2} + \beta_j \right) \\ &\quad \text{Si } SS_j \text{ es una fracción } \beta_j \text{ de } Q_j: \\ &= \frac{K_j}{T_j} + h_j \lambda_j T_j \left(\frac{1}{2} + \beta_j \right) \end{aligned}$$

Costos:

$$\begin{aligned} N &= \text{Cantidad de minoristas} + \text{Cantidad de bodegas} \\ &= \text{Cantidad de minoristas} + 1 \end{aligned}$$

★ **Costo de adquirir**

$$\sum_{j=0}^N \frac{K_j}{T_j}$$

★ **Costo de mantener**

$$\sum_{j=0}^N h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right)$$

★ **Costo total**

$$\sum_{j=0}^N \frac{n_j K_j}{T_0} + h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right)$$

Receta (caso determinístico): Suponiendo que tenemos λ, K y h de todos los minoristas y de la bodega.

- Poner h_j y λ_j en la misma temporalidad $\forall j$.
- Aplicar heurística de Shwarz: primero encontraremos parámetros iniciales

$T_j^* \forall j$ y luego los recalcularemos para llegar a los finales $T_j \forall j$

- 1) Encontrar todos los T_j^*

$$T_j^* = \sqrt{\frac{2K_j}{h_j \lambda_j}}$$

- 2) Armar conjunto J

$$J = \{j : T_j^* > T_0^*\}$$

- 3) Recalcular T_0

$$T_0 = \begin{cases} T_0^*, & \text{si } J = \emptyset \\ \sqrt{\frac{2(K_0 + \sum_{j \in J} K_j)}{\lambda_0 h_0 + \sum_{j \in J} \lambda_j h_j}}, & \text{si } J \neq \emptyset \end{cases}$$

- 4) Inicializar valor de $n_j \forall j$. El de la bodega siempre será 1 y no se deberá cambiar aún si hay mejores valores al hacer la optimización.
5) Poner función objetivo de costo total para cada j :

$$CT_j = \sum_{j=0}^N \frac{n_j K_j}{T_0} + h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right)$$

- 6) Solver para minimizar función de costos tal que $n_j^* \in \mathbb{N}$ y $n_j^* \geq 1$

- c) Calcular los $T_j \forall j$:

$$T_j = T_0 / n_j$$

- d) Calcular costo total: Este será la suma de los costos totales para cada j que calculamos anteriormente:

$$CT = \sum_{j=0}^N CT_j = \sum_{j=0}^N \frac{n_j K_j}{T_0} + h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right)$$

2.3. Ejercicios anidados

2.3.1. Caso 1-1-n con énfasis en 1-1

Receta:

- a) Poner toda la información que nos den en la misma temporalidad.

Parte 1-1:

- b) Identificar implicados en la parte de 1-1.
c) Encontrar demanda total: será la suma de las demandas de cada n de los minoristas:

$$\lambda = \sum_{j=1}^n \lambda_j$$

d) Reemplazar en la ecuación de costos:

$$Q_W = nQ_R$$

e) Identificar stocks de seguridad. Asumiremos que están dados por combinaciones lineales de Q_R , Q_W y λ :

$$SS_W = \alpha Q_R + \beta Q_W + \theta \lambda = Q_R(\alpha + \beta n) + \theta \lambda$$

$$SS_R = \gamma Q_R + \sigma Q_W + \delta \lambda = Q_R(\gamma + \sigma n) + \delta \lambda$$

$$\alpha, \beta, \gamma, \sigma, \theta, \delta \in [0, 1)$$

f) Derivar función de costos totales con respecto a Q_R y despejar Q_R :

$$Q_R^* = \sqrt{\frac{\lambda \left(\frac{K_W}{n} + K_R \right)}{i_W C_W \left(\frac{n}{2} + \alpha + \beta n \right) + i_R (C_R - C_W) \left(\frac{1}{2} + \gamma + \sigma n \right)}}$$

g) Dejar expresados T_R , Q_W en función de n^* y Q_R :

$$Q_R = \lambda T_R \Rightarrow T_R = \frac{Q_R}{\lambda}, \quad Q_W = nQ_R$$

h) Inicializar valor de n^* .

i) Expresar función de costos con los parámetros que dejamos expresados.

j) Solver para minimizar función de costos tal que $n^* \in \mathbb{N}$ y $n^* \geq 1$

$$CT(Q_R) = \frac{\lambda}{Q_R} \left(\frac{K_W}{n} + K_R \right) + Q_R \left[i_W C_W \left(\frac{n}{2} + \alpha + \beta n \right) + i_R (C_R - C_W) \left(\frac{1}{2} + \gamma + \sigma n \right) \right] + \lambda [i_W C_W \theta + i_R (C_R - C_W) \delta]$$

Parte 1-n:

9. Tomamos $T_R = T_0$ y retomamos la receta de 1-n desde el paso 4 (paso d en estas notas).

d) Inicializar valor de $n_j \forall j$. Ya vimos que $T_0 = T_R$.

e) Encontrar SS_j si lo hay

f) Poner función objetivo de costo total para cada j :

$$CT_j = \frac{n_j K_j}{T_0} + h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right)$$

g) Solver para minimizar función de costos tal que $n_j^* \in \mathbb{N}$ y $n_j^* \geq 1$

10. Encontramos T_j :

$$T_j = T_0/n_j = T_R/n_j$$

11. Encontramos Q_j :

$$Q_j = \lambda_j * T_j$$

12. Encontramos costos totales de la política:

$$\sum_{j=0}^N \frac{n_j K_j}{T_0} + h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right)$$

Si nos ponen a discriminar por costos:

- Costo de primer eslabón (bodega-warehouse):

$$C_W = \frac{K_W \lambda}{n Q_R} + i_W C_W \left(\frac{n Q_R}{2} + SS_W \right)$$

- Costo de segundo eslabón (centro de distribución – Retailers):

$$\frac{K_R \lambda}{Q_R} + i_R (C_R - C_W) \left(\frac{Q_R}{2} + SS_R \right)$$

- Costo de los demás eslabones (minoristas - j):

$$\frac{n_j K_j}{T_0} + h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right) \quad \forall j$$

2.3.2. Caso 1-1-n con énfasis en 1-n

Receta:

1. Poner datos del enunciado en misma temporalidad.

Parte 1-n (solo minoristas)

2. Aplicar heurística de Shwarz: primero encontraremos parámetros iniciales $T_j^* \forall j$ y luego los recalcularemos para llegar a los finales $T_j \forall j$

- a) Encontrar todos los T_j^*

$$T_j^* = \sqrt{\frac{2K_j}{h_j \lambda_j}}$$

- b) Armar conjunto J

$$J = \{j : T_j^* > T_0^*\}$$

- c) Recalcular T_0

$$T_0 = \begin{cases} T_0^*, & \text{si } J = \emptyset \\ \sqrt{\frac{2 \left(K_0 + \sum_{j \in J} K_j \right)}{\lambda_0 h_0 + \sum_{j \in J} \lambda_j h_j}}, & \text{si } J \neq \emptyset \end{cases}$$

- d) Inicializar valor de $n_j \forall j$.
 e) Poner función objetivo de costo total para cada j :

$$CT_j = \frac{n_j K_j}{T_0} + h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right)$$

- f) Solver para minimizar función de costos tal que $n_j^* \in \mathbb{N}$ y $n_j^* \geq 1$

3. Calcular los $T_j \forall j$:

$$T_j = T_0 / n_j$$

4. Calcular costo total de minoristas: Este será la suma de los costos totales para cada j que calculamos anteriormente, notemos que ahora T_0 corresponde al CD, no a la bodega.

$$CT = \sum_{j=1}^N CT_j = \sum_{j=1}^N \frac{n_j K_j}{T_0} + h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right)$$

Parte 1-1

5. Hacemos matriz de costos para los eslabones de 1-1

Nombre	n	T	Q	C. Orden	C. Mant
CD R	Inicializar	$T_R = T_0$	$\lambda_R * T_0$	K_R / T_0	$i_R * c_R * (Q_R / 2 + SS_R)$
Bodega W	1	$T_W = T_0 * n_R$	$Q_R * n_R$	K_W / T_W	$i_W * c_W * (Q_W / 2 + SS_W)$

6. Expresar costos totales sumando costos de la tabla
 7. Solver para encontrar n_R que minimice costos totales de 1-n.

2.3.3. Caso 1-n-1

Receta:

★ Parte 1-n

1. Identificamos parámetros básicos para todos los niveles de la cadena

Eslabón	K [\$/pedido]	c [\$/und]	i [%/Tiempo]	λ [unds/Tiempo]	SS [unds]
Copiamos de enunciado					

2. Ponemos todo en la misma temporalidad y en mismas unidades.
 3. Siempre operaremos yendo de distribuidores a distribuidos.
 4. Aplicamos heurística de Shwarz:

a) Encontrar todos los T_j^*

$$T_j^* = \sqrt{\frac{2K_j}{h_j\lambda_j}}$$

b) Armar conjunto J

$$J = \{j : T_j^* > T_0^*\}$$

c) Recalcular T_0

$$T_0 = \begin{cases} T_0^*, & \text{si } J = \emptyset \\ \sqrt{\frac{2(K_0 + \sum_{j \in J} K_j)}{\lambda_0 h_0 + \sum_{j \in J} \lambda_j h_j}}, & \text{si } J \neq \emptyset \end{cases}$$

d) Inicializar valor de $n_j \forall j$.

Nombre	n	T	Q	C. Orden	C. Mant
CD 1	Inicializar	$T_j = T_0/n$	$\lambda_R * T_j$	K_R/T_j	$i_R * c_R * (Q_R/2 + SS_R)$
⋮					

e) Poner función objetivo de costo total para cada j :

$$CT_j = \frac{n_j K_j}{T_0} + h_j \left(\frac{\lambda_j T_0}{2n_j} + SS_j \right)$$

f) Solver para minimizar función de costos tal que $n_j^* \in \mathbb{N}$ y $n_j^* \geq 1$

★ Parte 1-1

5. Pasamos al siguiente eslabón por cada fila de la anterior matriz que acabamos de crear. Se heredará el T_j del proveedor de cada minorista como T_0 para cada minorista y repetiremos la heurística desde la tabla final

a) Identificamos el T_j del proveedor de cada minorista. Este será el T_0 de cada minorista.

b) Creamos, llenamos y optimizamos la siguiente tabla para todos los minoristas encontrando el n óptimo.

Nombre	n	T	Q	C. Orden	C. Mant
M1	Inicializar	$T_j = T_0/n$	$\lambda_{M1} * T_j$	K_{M1}/T_j	$i_{M1} * c_{M1} * (Q_{M1}/2 + SS_{M1})$
⋮					

6. Repetir los pasos anteriores para todos los minoristas asociados a los establecimientos que los proveen.

Capítulo 3

Modo de transporte

Notación Modo de transporte

- λ : Demanda [unds/Tiempo]
- r : Flete transporte [\$/unds]
- τ : Tiempo de tránsito [Tiempo]
- T : Tiempo de ciclo de aprovisionamiento [Tiempo]
- f : Frecuencia de ciclo de aprovisionamiento [1/Tiempo]
- Q : Tamaño de la orden [unds]
- SS_k : Stock de seguridad en el eslabón k de la cadena [unds]

Ecuaciones Modo de transporte

$$C' = C + r$$

Receta:

Indexaremos los modos de transporte en $i \in I = \{1, 2, \dots, n\}$ y los modos de carga en $j \in J = \{1, 2, \dots, m\}$

1. Poner todo en misma temporalidad y en unidades comparables con factores de conversión que nos da el enunciado.
2. Encontrar tasa de consumo λ .
3. Encontrar tiempo de reaprovisionamiento T .
4. Encontrar tamaño de lote teniendo en cuenta el modo de transporte

$$Q = \lambda/T$$

5. PASO MÁS IMPORTANTE: Identificar qué costos están por unidad, por unidad de transporte/carga o por pedido. Usaremos la siguiente notación:

- P : Costos totales por pedido [\$/pedido].
- K : Costos totales por método de transporte o método de carga [\$/und transporte] [\$/und carga]
- U : Costos totales por unidad [\$/und]

6. Crear combinaciones entre métodos de transporte y carga y encontrar costos unitarios para encontrar r

i	j	Q	$\#_i$	$\#_j$	Costos totales	τ total	r
1	1	Copiamos	Eq. 3.1	Eq. 3.2	Eq. 3.3	Eq. 3.4	Eq. 3.5
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

$$\#_i = \text{REDONDEAR.MAS} \left(\frac{Q}{\text{Capacidad}_i}; 0 \right) \quad (3.1)$$

$$\#_j = \text{REDONDEAR.MAS} \left(\frac{Q}{\text{Capacidad}_j}; 0 \right) \quad (3.2)$$

$$\text{Costos totales}_{ij} = (P_i + P_j) + (K_i * \#_i + K_j * \#_j) + (U_i + U_j) * Q \quad \forall (i, j) \in I \times J \quad (3.3)$$

$$\tau_{ij} = \tau_i + \tau_j \quad (3.4)$$

$$r_{ij} = \frac{\text{Costos totales}_{ij}}{Q} = \frac{\text{Costos totales}_{ij}}{T * \lambda} \quad (3.5)$$

7. Encontrar parámetros de costos para todos los niveles de la cadena. La mayoría de veces nos los dan, pero incluimos fórmulas por si hay que despejar

★ **Planta:**

Parámetro	Fórmula
i_{Pl}	h_{Pl} / C_{Pl}
C_{Pl}	h_{Pl} / i_{Pl}
SS_{Pl}	Enunciado

★ **Tránsito:**

Parámetro	Fórmula
i_{Tr}	h_{Tr} / C_{Tr}
C_{Tr}	h_{Tr} / i_{Tr}
SS_{Tr}	Enunciado

★ **CD:**

Parámetro	Fórmula
i_{CD}	h_{CD} / C_{CD}
C_{CD}	$h_{CD} / i_{CD} + r$
SS_{CD}	Enunciado

8. Tabla de costos

i	j	C.M.Pl	C.M.Tránsito	C.Transporte	C.M CD	CT
1	1	Eq. 3.6	Eq. 3.7	Eq. 3.8	Eq. 3.9	Suma

$$C.M.Pl_{ij} = i_{Pl} * C_{Pl} * \left(\frac{Q}{2} + SS_{Pl} \right) \quad (3.6)$$

$$C.M.Tránsito_{ij} = i_{Tr} * C_{Tr} * \lambda * \tau_{ij} \quad (3.7)$$

$$C.Transporte_{ij} = \frac{1}{T} * (\text{Costos totales})_{ij} = \lambda * r_{ij} \quad (3.8)$$

$$C.M CD_{ij} = i_{CD} * (C_{CD} + r_{ij}) * (Q/2 + SS_{CD}) \quad (3.9)$$

Capítulo 4

El producto en la logística

Notación curva 80-20

- i : Índice de productos.
- n : Número total de productos, tendremos $i = 1, 2, \dots, n$
- X_i : Porcentaje acumulado de número de productos [%]
- $Y_i \text{ teórico}$: Porcentaje acumulado de ventas encontrado mediante la fórmula [%]
- $Y_i \text{ real}$: Porcentaje acumulado de ventas encontrado por medio del análisis de datos reales [%]
- A : Constante que mide el grado de desigualdad entre la cantidad de ítems y su contribución acumulada a las ventas en la curva de Pareto.
- Clasificación:
 - A: Los productos que componen la mayor parte de las ventas teniendo la menor presencia ($Y \approx 80\% - X \approx 20\%$)
 - B: Los productos que componen $X \approx 20\% - 50\%$ del total de productos.
 - C: Los productos que quedan y acumulan el restante.

Ecuaciones curva 80-20

★ **Parámetros curva:**

Nota: Asumimos que los productos están organizados de mayor a menor proporción de ventas tranzadas.

$$Error_i = Y_i \text{ teorico} - Y_i \text{ real}$$

$$X_i = \frac{i}{n}, \quad i = 1, 2, \dots, n$$

$$Y_i \text{ teorico} = \frac{(1 + A)X_i}{X_i + A} = Error_i + Y_i \text{ real}$$

$$Y_i \text{ real} = \text{Porcentaje de ventas de producto } i = Y_i \text{ teorico} - Error$$

$$A_i = \frac{X_i(1 - Y_i)}{Y_i - X_i}$$

Nota: Muchas veces nos dicen que un solo A aplica para todos los productos para poder simplificar cálculos.

★ **Ventas:**

$$(\text{Ventas acumuladas})_i = (Y \text{ real})_i * (\text{Ventas totales})$$

$$(\text{Ventas del producto } i) = \begin{cases} (\text{Ventas acumuladas})_i & \text{si } i = 1 \\ (\text{Ventas acum})_i - (\text{Ventas acum})_{i-1} & \text{si } i > 1 \end{cases}$$

Receta: Caso general donde nos dan lista de productos y ventas asociadas a cada uno de la forma:

Producto	Ventas
P_1	V_1
P_2	V_2
$\vdots$	$\vdots$
P_n	V_n

La receta se reduce a llenar la siguiente tablita, en donde los productos estarán ordenados de mayor a menores ventas:

Producto	Ventas	i	%	Y_R	X	Class	A_j	Y_T	Error
P1	V1	1	$\frac{V1}{\sum V_i}$	% ₁	$\frac{1}{n}$	Eq. 4.1	Eq. 4.2	Eq. 4.3	Enunciado
P2	V2	2	$\frac{V2}{\sum V_i}$	% ₁ + % ₂₋₁	$\frac{2}{n}$	Eq. 4.1	Eq. 4.2	Eq. 4.3	Enunciado
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
Pn	Vn	n	$\frac{Vn}{\sum V_i}$	% _{0n} + % _{0n-1}	$\frac{n}{n}$	Eq. 4.1	Eq. 4.2	Eq. 4.3	Enunciado

$$\text{Class}_i = \begin{cases} A & \text{si } X_i \leq 20 \% \\ B & \text{si } 20 \% < X_i \leq 50 \% \\ C & \text{si } X_i > 50 \% \end{cases} \quad (4.1)$$

$$A_i = \frac{X_i(1 - Y_i)}{Y_i - X_i} \quad (4.2)$$

$$Y_i \text{ teorico} = \frac{(1 + A)X_i}{X_i + A} \quad (4.3)$$

Donde A minimiza el promedio de los errores, podemos hacerlo con solver

Receta: Casos escalados

No hay receta, los ejercicios pueden variar tanto que tenemos que apoyarnos en las definiciones, ecuaciones y tips del enunciado para encontrar valores específicos que nos pidan.

Capítulo 5

Ruta más corta

5.1. Algoritmo Dijkstra

¿Qué debo tener claro para aplicar Dijkstra en logística?

Antes de resolver ejercicios usando el algoritmo de Dijkstra con restricciones, asegúrate de comprender y poder responder lo siguiente:

■ Sobre el grafo y su estructura:

- ¿Es el grafo dirigido o no dirigido? ($G = (V, E)$)
- ¿Qué representan los nodos (V) y las aristas (E) en el contexto?
- ¿Cuáles son los atributos relevantes por arista (costo, tiempo, capacidad, distancia, etc.)?

■ Sobre los atributos acumulables:

- ¿Qué atributos vas a acumular? (Ej: costo, tiempo, distancia)
- ¿Cuál es el criterio principal de optimización? (Ej: minimizar costo)
- ¿Qué atributos se usarán como desempate? (Ej: tiempo)

■ Sobre las restricciones:

- ¿Qué restricciones fuertes existen? (Ej: "no usar vuelos después de las 11am")
- ¿Qué restricciones débiles o penalizaciones se deben aplicar? (Ej: -15 % al costo si carga > capacidad)
- ¿Dependen de valores como carga, horario, o modo de transporte?

■ Sobre el output esperado:

- ¿Qué debe devolver tu función? ¿Solo la ruta, o también tiempo, distancia, costo?
- ¿Cómo presentarás los resultados de forma legible para validación y análisis?

Cajita de herramientas networkx

1. Creación de grafos

- `G = nx.Graph()` (No dirigido)
- `G = nx.DiGraph()` (Dirigido)

2. Agregar nodos y aristas

- `G.add_node(n)` Añade un nodo
- `G.add_nodes_from([n1, n2, ...])`
- `G.add_edge(u, v)` Añade arista entre u y v
- `G.add_edge(u, v, weight=w, carac1=c1, ...)` Con más características
- `G.add_edges_from([(u,v), (x,y)])`

3. Acceso a nodos y aristas

- `G.nodes` Lista de nodos
- `G.edges` Lista de aristas
- `G.neighbors(n)` Vecinos del nodo n
- `G[u][v]['weight']` Peso de la arista (u, v)

4. Algoritmos

- `nx.dijkstra_path(G, source, target)` Camino más corto
- `nx.dijkstra_path_length(G, source, target)`
- `nx.shortest_path(G)` (No ponderado)
- `nx.minimum_spanning_tree(G)` Árbol de expansión mínima

5. Visualización y atributos

- `nx.draw(G, with_labels=True)` Visualizar
- `nx.set_edge_attributes(G, values, 'weight')`

6. Lectura y escritura

- `nx.read_edgelist('archivo.txt')`
- `nx.write_edgelist(G, 'archivo.txt')`
- `nx.from_pandas_edgelist(df, 'origen', 'destino', edge_attr='peso')`

Receta: Dijkstra con restricciones en grafo dirigido.

Para esta receta manejaremos código parte por parte. Para simplificación de la misma,

solo explicaremos las partes que se deben modificar del código dado que las restricciones serán tratadas de forma diferente en cada caso, dependiendo del Excel que nos den y el tipo de restricción. Para esto, le pondremos un fondo **verde** para los bloques que se pueden copiar y pegar directamente y un fondo **naranja** para los que deben editarse así sea un poco.

En su caso más simple, este algoritmo es una función que toma una tupla (Grafo, NodoInicial, NodoFinal) y devuelve una secuencia de los nodos que componen la ruta más corta.

En este ejemplo, nos dicen que transportamos una carga y nos piden que demos el tiempo, el costo y la secuencia de la ruta más corta sujeto a varias condiciones que exploraremos más adelante.

1. Librerías básicas

```
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt
import heapq
```

2. Lectura de archivo y formateo de las dos primeras hojas a dataframes:

```
archivo = "/Users/.../Downloads/Plantilla - Copy.xlsx"
df1 = pd.read_excel(archivo, sheet_name=0)
df2 = pd.read_excel(archivo, sheet_name=1)
```

Nota: En este ejemplo, solo usaremos un dataframe llamado info

★ Sección 1: Creación de grafo

3. Creamos grafo dirigido

```
G = nx.DiGraph()
```

4. Iteramos sobre las filas del dataframe que queramos y asignamos a variables todos los atributos que nos puedan convenir. Este es solo un ejemplo:

```
for _, fila in info.iterrows():
    origen = fila['Origen']
    destino = fila['Destino']

    # Extraer valores base
    distancia = fila['Distancia']
    velocidad = fila['Velocidad']
    frontera = fila['Frontera']
    alta_demanda = fila['Alta_Demanda']
    capacidad = fila['Capacidad']
    horario = fila['Horario']
    terrestre = fila['Terrestre']
    aereo = fila['Aereo']
    vuelos = fila['Vuelos']
```

5. Calculamos valores derivados de estas variables que acabamos de asignar. Esto es útil cuando disponemos de velocidades y distancias para encontrar tiempos, o funciones de costos, por ejemplo:

```
# Calcular valores derivados
costo = 15 * distancia + 200 * frontera + (150 if
    alta_demanda == 1 else 0)
tiempo = distancia / velocidad if velocidad > 0 else
    float('inf')
```

6. Creamos diccionario de atributos y los añadimos a la arista correspondiente

```
atributos = {
    "distancia": distancia,
    "velocidad": velocidad,
    "frontera": frontera,
    "alta_demanda": alta_demanda,
    "capacidad": capacidad,
    "horario": horario,
    "terrestre": terrestre,
    "aereo": aereo,
    "vuelos": vuelos,
    "costo": costo,
    "tiempo": tiempo
}

G.add_edge(origen, destino, **atributos)
```

7. Visualización de grafo: opcional. La única parte que tenemos que actualizar, es el “costo” que deberá tener el nombre de un atributo presente en el diccionario anterior.

```
mostrar = True
if mostrar:
    plt.figure(figsize=(10, 6))
    pos = nx.spring_layout(G, seed=42)
    nx.draw(G, pos, with_labels=True, node_size=700,
        node_color='lightblue', font_weight='bold')
    edge_labels = {
        (u, v): f"{int(d['costo'])} COP"
        for u, v, d in G.edges(data=True)
    }
    nx.draw_networkx_edge_labels(G, pos, edge_labels=
        edge_labels, font_size=8)
    plt.title("Grafo con costos")
    plt.axis('off')
    plt.show()
```

★ Sección 2: Construcción de camino usando Dijkstra

8. Identificamos los parámetros que son constantes a lo largo de la función: siempre se tendrán como mínimo el `NodoInicial` y el `NodoFinal`, pero para este ejemplo, consideraremos también la carga

```
def dijkstra_modular(G, inicio, destino, carga):
```

9. Identificamos los valores que queremos acumular o actualizar en cada iteración del método, en este caso, como en este ejemplo nos piden el tiempo, distancia y el costo, inicializaremos una cola de prioridad (heap) y los valores de estos en infinito para todos los nodos:

```
heap = [] # (costo acumulado, tiempo acumulado,
           distancia, nodo actual)
cost = {n: float('inf') for n in G.nodes}
tiempo = {n: float('inf') for n in G.nodes}
dist = {n: float('inf') for n in G.nodes}
ruta = {n: [] for n in G.nodes}
```

Nota: El diccionario `ruta[]` estará presente en todos lados pero nunca lo tocaremos ni editaremos nada relacionado a él. Este solamente actualizará el camino óptimo que el algoritmo seguirá.

10. Inicializamos parámetros a actualizar según enunciado: en este ejemplo, empezamos a las 00:00 de la mañana el trayecto, en el km 0 y el costo inicial es de \$0 y los insertamos en la cola de prioridad.

```
cost[inicio] = 0
tiempo[inicio] = 0
dist[inicio] = 0
ruta[inicio] = [inicio]
heapq.heappush(heap, (cost[inicio], tiempo[inicio],
                     dist[inicio], inicio))
```

11. Creamos ciclo principal que va descartando nodos y acumulando atributos definidos anteriormente. El nodo actual será llamado `arista` por el resto de la iteración:

```
while heap:
    costo_actual, tiempo_actual, dist_actual, actual =
        heapq.heappop(heap)

    if actual == destino:
        break
```

12. Creamos ciclo secundario que va identificando los vecinos y evaluando el óptimo entre ellos

```
for vecino in G.successors(actual):
    arista = G[actual][vecino]
```

13. Por cada vecino y cada arista, debemos identificar si este es accesible desde la arista actual evaluando dos tipos de restricciones:

- a) Identificamos restricciones débiles: aquellas que penalizan ciertas situaciones pero no hacen accesible el nodo vecino. En este ejemplo, si la carga excede la capacidad de la arista actual, el costo aumenta:

```
#Restricciones debiles y actualizacion
arista = G[actual][vecino]
costo_base = arista['costo']
if carga > arista['capacidad']:
    costo_base *= 1.15
```

Este proceso también conlleva a recalcular los atributos que vamos a ir actualizando en cada iteración del método dada su acumulación. En este ejemplo, se actualiza el tiempo, la distancia y el costo

```
nuevo_costo = costo_actual + costo_base
nuevo_tiempo = tiempo_actual + arista['
    tiempo']
nueva_dist = dist_actual + arista['
    distancia']
```

- b) Identificamos restricciones fuertes: aquellas que si no se cumplen vuelven inaccesible el nodo vecino que se está estudiando. En este ejemplo, no se usarán los caminos aéreos que estén disponibles después de las 11am. El continue que sale al final simplemente salta la iteración del vecino actual y procede a evaluar el vecino siguiente en lista. No confundir con el break, que cerraría todo el ciclo y no visitaría más vecinos.

```
# Restricciones fuertes
if arista['aereo'] == 1 and arista['horario
    '] > 11:
    continue
```

14. Finalmente, vemos si en realidad se consigue una mejora al actualizar la lista actual de vecinos. En este ejemplo, miramos si el nuevo costo actualizado para el vecino es menor que el actual y usamos como criterio de desempate el tiempo acumulado que tienen las rutas candidatas.

```
if (
    nuevo_costo < cost[vecino]
    or (nuevo_costo == cost[vecino] and
        nuevo_tiempo < tiempo[vecino]))
):
    cost[vecino] = nuevo_costo
    tiempo[vecino] = nuevo_tiempo
    dist[vecino] = nueva_dist
    ruta[vecino] = ruta[actual] + [vecino]
    heapq.heappush(heap, (nuevo_costo,
        nuevo_tiempo, nueva_dist, vecino))
```

Como todos los parámetros de interés están debidamente acumulados, retornamos los valores de interés. Ojo que estos vendrán en forma de tupla

```
if not ruta[destino]:
    return None, float('inf'), float('inf'), float('inf')

return ruta[destino], dist[destino], tiempo[destino],
        cost[destino]
```

15. Finalmente, ejecutamos la función con los valores iniciales que precisamos e imprimimos los resultados de forma amigable para que nos evalúen más fácilmente

```
#Ejecutaremos la funcion con nombres y valores de carga
    propios del problema
resultado = dijkstra_modular(G, "San Diego", "Denver",
    carga=20)

print("RESULTADOS:")
print(f"Ruta optima: {resultado[0]}")
print(f"Distancia total recorrida: {resultado[1]} km")
print(f"Tiempo total de ruta: {resultado[2]} h")
print(f"Costo total: ${resultado[3]}")
```

Receta: Pseudocódigo del ejemplo anterior**Input:**

G: grafo dirigido con atributos en cada arista
inicio: nodo origen
destino: nodo destino
carga: cantidad a transportar

Inicializar:

Para cada nodo n :

costo[n] $\leftarrow \infty$
tiempo[n] $\leftarrow \infty$
distancia[n] $\leftarrow \infty$
ruta[n] $\leftarrow$ lista vacía

costo[inicio] $\leftarrow 0$
tiempo[inicio] $\leftarrow 0$
distancia[inicio] $\leftarrow 0$
ruta[inicio] $\leftarrow$ [inicio]
heap $\leftarrow (0, 0, 0, \text{inicio})$

Mientras heap no esté vacío:

(costo_actual, tiempo_actual, distancia_actual, nodo_actual) $\leftarrow$ extraer mínimo del heap

Si nodo_actual == destino: salir del ciclo

Para cada vecino de nodo_actual:

arista $\leftarrow$ G[nodo_actual][vecino]

Restricción fuerte:

Si arista.aéreo == 1 y arista.horario > 11: **continuar**

costo_base $\leftarrow$ arista.costos

Restricción débil:

Si carga > arista.capacidad:

costo_base $\leftarrow$ costo_base $\times$ 1.15

nuevo_costo $\leftarrow$ costo_actual + costo_base

nuevo_tiempo $\leftarrow$ tiempo_actual + arista.tiempo

nueva_distancia $\leftarrow$ distancia_actual + arista.distancia

Criterio de mejora:

Si nuevo_costo < costo[vecino] o

(nuevo_costo == costo[vecino] y nuevo_tiempo < tiempo[vecino]):

Actualizar costo[vecino], tiempo[vecino], distancia[vecino]

ruta[vecino] $\leftarrow$ ruta[nodo_actual] + [vecino]

insertar en heap: (nuevo_costo, nuevo_tiempo, nueva_distancia, vecino)

Fin del ciclo

Si ruta[destino] está vacía:

No existe ruta válida

Output:

ruta[destino], distancia[destino], tiempo[destino], costo[destino]

Como podemos ver, esto es bastante extenso, por ende desarrollaremos un bosquejo que nos facilite las secciones más generales del algoritmo y solo tengamos que actualizar las restricciones y parámetros propios del enunciado según nos digan, estos estarán expresados en color **Naranja**:

Receta: Generalización pseudocódigo

Input:

G: grafo dirigido con atributos por arista

s: nodo origen

t: nodo destino

ATRIBUTOS: lista de atributos acumulables (ej. *costo, tiempo, distancia*)

RESTRICCIONES: conjunto de restricciones fuertes y débiles

FACTORES: transformaciones o penalizaciones por condiciones específicas

Inicializar:

Para cada nodo n :

$A[n][\text{atributo}] \leftarrow \infty$ para cada atributo

$\text{ruta}[n] \leftarrow$ lista vacía

$A[s][\text{atributo}] \leftarrow 0$ para cada atributo

$\text{ruta}[s] \leftarrow [s]$

$\text{heap} \leftarrow (\text{valores iniciales}, s)$

Mientras heap no esté vacío:

$(\text{valores acumulados}, \text{nodo_actual}) \leftarrow$ extraer mínimo del heap

Si $\text{nodo_actual} == t$: salir del ciclo

Para cada vecino de nodo_actual :

$\text{arista} \leftarrow G[\text{nodo_actual}][\text{vecino}]$

Aplicar restricciones fuertes:

Si alguna condición en **RESTRICCIONES fuertes** se viola:

continuar al siguiente vecino

Aplicar restricciones débiles / penalizaciones:

Modificar atributos de arista según **FACTORES**

Calcular nuevos valores acumulados:

Para cada atributo: $\text{nuevo} \leftarrow \text{actual} + \text{modificado}$

Evaluar criterio de mejora:

Si $\text{nuevo}[\text{atributo principal}] < A[\text{vecino}][\text{atributo principal}]$

o empate con menor valor en **atributo secundario:**

Actualizar $A[\text{vecino}][\text{atributos}]$

$\text{ruta}[\text{vecino}] \leftarrow \text{ruta}[\text{nodo_actual}] + [\text{vecino}]$

push en heap: $(\text{valores}, \text{vecino})$

Fin del ciclo

Si $\text{ruta}[t]$ está vacía:

No existe ruta válida

Output:

$\text{ruta}[t]$, $A[t][\text{atributo}]$ para cada atributo acumulado

Capítulo 6

Ruteo

6.1. Generalización de heurística Clark and Wright

Notación generalización de heurística Clark and Wright

- C : Conjunto de clientes.
- i : Índice de cliente perteneciente al conjunto de clientes $i \in C$
- j : Índice secundario de cliente perteneciente al conjunto de clientes $j \in C$

Receta: Usaremos el método matricial para esta receta. Para generalizarla a cualquier tipo de parámetro que nos den, es posible que manejemos más matrices para acumular los atributos que nos indique el enunciado.

1. Creamos una ruta para todo cliente $i \in C$ con origen y retorno al depósito. Tendremos $|C|$ rutas en este primer paso de inicialización:

$$0 - i - 0 \quad \forall i \in C$$

2. Identificamos matriz de distancias, esta siempre será necesaria en los ejercicios puesto que el algoritmo está diseñado para minimizar distancias.
3. Calculamos la matriz de ahorros netos. Manejaremos una notación de distancias entre cliente i (filas) y cliente j (columnas) como $d[i, j]$.

Para hacer matriz de forma fácil: Notemos que cada ahorro se ve de la forma $A[i, j] = d[i, 0] + d[0, j] - d[i, j]$, por ende, la matriz de ahorros se verá de la forma

i/j	0	1	2	3	...	n
0	0	0	0	0	...	0
1	0	0	$A[1,2]$	$A[1,3]$	...	$A[1,n]$
2	0	$A[2,1]$	0	$A[2,3]$	...	$A[2,n]$
3	0	$A[3,1]$	$A[3,2]$	0	...	$A[3,n]$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
n	0	$A[n,1]$	$A[n,2]$	$A[n,3]$	...	0

Así pues, podemos hacer una fórmula arrastrable en Excel tal que

$$A[i, j] = d[\$i, j] + d[i, \$j] - d[i, j]$$

Donde \$i y \$j representa el fijar las coordenadas de i y j. Por ende,

a) Inicializamos $A[0, 0] = d[\$0, 0] + d[0, \$0] - d[0, 0]$

b) Arrastramos

c) Llenamos diagonal con ceros, o hacemos condicional para que esto se cumpla.

4. Considerar celda $A[i, j]$ no tachada con mayor ahorro disponible y verificar que se cumplen las siguientes condiciones

a) Los nodos i y j no están en la misma secuencia.

b) Existen las conexiones $(i, 0)$ y $(0, j)$.

c) No se violan las restricciones propias del problema.

★ **Chequeo de restricciones propias via simulación por eventos:** Cuando tenemos restricciones de capacidad, ventanas de tiempo, autonomía de vehículos, etc. es necesario ver en cada llegada o trayecto se respeten las condiciones impuestas.

Para esto, haremos una tablita donde las filas estarán indexadas por cliente dentro de la ruta y las columnas serán el atributo que tengamos que acumular o tener en cuenta mientras actualizamos las rutas. En un ejemplo escalado donde tenemos que tener en cuenta capacidad de carga, autonomía de vehículos, tiempo de servicio y ventanas de tiempo asumiendo espera, la tabla quedará algo como:

Índice	i	Distancia	H. Salida	T. transporte	H. Llegada	T. Espera	T. servicio
1	C1	D1	HS1	TT1	HL1 = TT1+TE1	TE1	TS1
⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮

5. Proceder dependiendo de si se cumplen las condiciones:

- **Todas se cumplen:** Agregar arco (i, j) , actualizar conexiones $(i, 0)$ y $(0, j)$, y tachar fila y columna de matriz de ahorro. Estas celdas tachadas no serán tenidas en cuenta más adelante.
- **Por lo menos una no se cumple:** Tachar solamente la celda (i, j) que se está evaluando.

6. Volver al paso 4 y repetir hasta que hayamos tachado todas las posiciones de la matriz.

Nota: En caso de que unamos dos rutas existentes, tendremos que reconstruir la simulación completa de ruteo teniendo en cuenta este nuevo escenario.

Receta: Generalización pseudocódigo

1. Inicialización

Para cada cliente i , crear una ruta individual: $R_i = [0, i, 0]$.

2. Cálculo de ahorros

Para cada par de clientes $i \neq j$, calcular:

$$S_{ij} = c_{0i} + c_{0j} - c_{ij}$$

3. Ordenar ahorros

Crear una lista ordenada $S = [(i, j, S_{ij})]$ según S_{ij} de mayor a menor.

4. Iterar sobre pares ordenados

Mientras existan elementos en S :

- Tomar el siguiente par (i, j) .
- Identificar rutas R_i y R_j .
- Verificar si:
 - i está al final de su ruta.
 - j está al inicio de su ruta.
 - $R_i \neq R_j$.
- **Evaluar restricciones fuertes:**
Rechazar la fusión si:
 - i y j ya están en la misma ruta
 - No existen las conexiones $(i, 0)$ o $(0, j)$.
 - [Editar] Restricción de precedencia u orden
 - [Editar] Demanda total excede capacidad
- **Evaluar restricciones débiles (penalizaciones):**
 - [Editar] Se viola ventana de tiempo
 - [Editar] Zona con penalización adicional

Ajustar S_{ij} o cancelar la fusión.

- Si la fusión es factible:
 - Unir R_i y R_j en una nueva ruta R_{ij} .
 - Eliminar de S todos los pares que involucren i o j .

5. Finalizar

Devolver el conjunto final de rutas como solución heurística.

6.2. Macro algoritmo Clark and Wright

Notación parámetros de visualización y ejecución

- **Bing Maps Key:** Código opcional para geolocalizar direcciones, calcular distancias y visualizar mapas. Se pega en la celda C2.
- **Number of depots:** Número de depósitos desde donde salen los vehículos.
- **Number of customers:** Número de clientes a atender (no incluye depósitos).
- **Distance computation method:**
 - Manual entry
 - Euclidean / Rounded Euclidean
 - Manhattan (Hamming)
 - Bird's flight (km / miles)
 - Bing Maps driving (km / miles)
- **Duration computation method:**
 - Manual entry
 - Average vehicle speed
 - Bing Maps driving durations
- **Bing Maps route type:**
 - Shortest
 - Fastest
 - Fastest – Real Time Traffic
- **Average vehicle speed:** Velocidad media usada para estimar duración cuando no se usa Bing Maps.
- **Number of vehicle types:** Número de tipos de vehículos con distinta capacidad, costo, etc.
- **Do vehicles return to depot?:**
 - Yes – only once (ruteo clásico)
 - No (open VRP)
 - Yes – may do so multiple times (multi-trip VRP)

Notación parámetros de visualización y ejecución

- **Time window type:**
 - Hard (visitas fuera de ventana son inválidas)
 - Soft (permitidas con penalización)
- **Backhauls?:** Indica si hay recolección luego de entregas. Útil para VRPs con pickups y deliveries.
- **Visualization background:**
 - Bing Maps
 - Blank
- **Location labels:** Qué mostrar sobre cada punto en el mapa (ID, nombre, ventana, etc.)
- **Warm start:** Si se activa, la macro usa la solución existente como punto de partida.
- **Show progress on status bar:** Si está en Yes, muestra iteración y costo parcial abajo en Excel.
- **CPU time limit:** Límite de tiempo (en segundos) para la ejecución de la macro de solución.

¿Para qué usamos la macro?

Respuesta sencilla: Porque es más fácil, rápido y no necesitamos estar pendientes de embarrarla en alguna iteración.

Entonces, ¿por qué no siempre usamos la macro?

Porque no consideramos reglas de desempate ni algunos atributos rebuscados de problemas. Por ejemplo, costos aumentados a partir de cierta distancia recorrida.

Entonces, si resolvemos un problema compatible con la macro de ambas formas ¿Deberíamos llegar a la misma respuesta, misma ruta y mismos costos acumulados totales?

Sí, siempre y cuando no hayan reglas de desempate.

Capítulo 7

Localización de instalaciones

7.1. Modelos basados en distancias

Estos modelos buscan encontrar el punto óptimo (x^*, y^*) para ubicar una instalación, minimizando una función objetivo basada en las distancias ponderadas a un conjunto de puntos (clientes, almacenes, etc.).

7.1.1. Método de la gravedad

Este método minimiza la suma de las **distancias euclidianas al cuadrado** entre el punto a ubicar y cada punto dado, ponderadas por un peso w_i .

Función objetivo

$$f(x, y) = \sum_i w_i \left[(x - a_i)^2 + (y - b_i)^2 \right]$$

Óptimo

La solución analítica es:

$$x^* = \frac{\sum_i w_i a_i}{\sum_i w_i}, \quad y^* = \frac{\sum_i w_i b_i}{\sum_i w_i}$$

Interpretación: Es el centro de masa o centroide ponderado de los puntos.

Receta

1. Organizar la información como tripletas (a_i, b_i, w_i) .
2. Calcular:

$$x^* = \frac{\sum w_i a_i}{\sum w_i}, \quad y^* = \frac{\sum w_i b_i}{\sum w_i}$$

3. Evaluar el valor objetivo si se desea comparar alternativas.

7.1.2. Método de Weiszfeld

Este método busca minimizar la **suma de las distancias euclidianas** reales, ponderadas por w_i .

Función objetivo

$$f(x, y) = \sum_i w_i \sqrt{(x - a_i)^2 + (y - b_i)^2}$$

Problema: No tiene solución cerrada. Se resuelve con un algoritmo iterativo.

Algoritmo de Weiszfeld

Dado un punto inicial $(x^{(0)}, y^{(0)})$, se actualiza:

$$x^{(l+1)} = \frac{\sum_i \frac{w_i a_i}{\sqrt{(x^{(l)} - a_i)^2 + (y^{(l)} - b_i)^2}}}{\sum_i \frac{w_i}{\sqrt{(x^{(l)} - a_i)^2 + (y^{(l)} - b_i)^2}}}$$

$$y^{(l+1)} = \frac{\sum_i \frac{w_i b_i}{\sqrt{(x^{(l)} - a_i)^2 + (y^{(l)} - b_i)^2}}}{\sum_i \frac{w_i}{\sqrt{(x^{(l)} - a_i)^2 + (y^{(l)} - b_i)^2}}}$$

Condición de paro: Error entre iteraciones menor a tolerancia ϵ .

Receta

1. Calcular el centro de gravedad como punto inicial.
2. Repetir el paso iterativo hasta convergencia.
3. Asegurarse que el punto solución no coincida con algún punto (a_i, b_i) , ya que el denominador se anula.

Nota: Este método se conoce como el *problema de Weber* o *problema del Fermat ponderado*.

Capítulo 8

Diseño de ruta

8.1. Algoritmo ADD

Notación algoritmo ADD

- i : i -ésimo cliente candidato a ser un centro de distribución.
- j : j -ésimo cliente cuya demanda debemos satisfacer.
- C : Conjunto de todos los clientes, $i, j \in C$, diremos que $|C| = n$.
- CV_{ij} : Costo variable del nodo i si se convirtiera en centro de distribución atendiendo al nodo j como cliente.
- CF_i : Costo fijo de abrir el nodo i como centro de distribución.

Receta: Caso básico teniendo en cuenta costos fijos y variables.

1. Creamos matriz de clientes y posibles centros de distribución asignando costos variables y fijos correspondientemente. En este caso, asumiremos que todos los nodos pueden convertirse en centros de distribución:

i/j	1	2	...	$\sum CV_j$	CF	CT
1	CV_{11}	CV_{12}	...	$\sum CV_j$	CF_1	$CT_1 = CF_1 + \sum CV_j$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

2. Empezamos a iterar, identificando el posible CD i con el menor costo total asociado, notaremos este nodo como $k^* \in C$.
3. Copiamos y pegamos tabla, poniendo como 0 la columna correspondiente al cliente óptimo del paso anterior $CV_{ik^*} = 0$ por medio de la siguiente actualización de fórmulas:

$$CV_{ij} = \min\{CV_{ij}, CV_{k^*j}\}$$

$$CF_i = CF_i + CF_{k^*}$$

Esta última fórmula significa que vamos acumulando costos fijos conforme vamos abriendo nuevos CDs.

4. Crear matriz auxiliar para chequear restricciones. En estas notas se encuentra desarrollada un ejemplo de cómo hacer para el algoritmo DROP. No obstante, se recomienda desarrollar una versión propia que quede fácil de memorizar acorde a la forma de aproximarse a los datos de cada quién.
5. Repetimos pasos 2 y 3 hasta que el mínimo CT aumente o hasta que asignemos los CDs que nos pide el enunciado.
6. Ya con los centros de distribución elegidos, copiamos y pegamos las filas de costos variables correspondientes a estos. Para cada columna indexada en j elegiremos el mínimo teniendo en cuenta reglas de desempate (si las hay). De esta forma, encontraremos qué CD abastece a qué cliente.

Receta: Pseudocódigo ADD**Pseudocódigo – Algoritmo ADD (Parte 1/2)****Inicialización:**

- Cargar matriz de distancias o costos M entre todos los nodos.
- Inicializar conjunto de centros de distribución (CDs): $S = \emptyset$.
- Inicializar variables asociadas a restricciones (por ejemplo, capacidad cubierta, clientes atendidos, etc.).
- Definir criterios de parada:
 - [EDITAR] Restricción de viabilidad del enunciado (ej: número máximo de CDs abiertos, capacidad total, cobertura geográfica, etc.).
 - Minimización del costo total de la solución.

Paso 1 – Evaluación Inicial:

- Para cada nodo j , calcular el [EDITAR] costo total de operar solo con j como CD.
- Seleccionar $j^* = \arg \min_j [\text{EDITAR}] \text{CostoTotal}(j)$.
- Añadir j^* a S .
- Eliminar fila y columna j^* de la matriz M .
- [EDITAR] Actualizar matrices auxiliares y variables de restricciones con base en la inclusión de j^* .

Pseudocódigo – Algoritmo ADD (Parte 2/2)

Paso 2 – Iteraciones hasta convergencia:

- Mientras no se cumpla **ningún** criterio de parada:
 - Para cada nodo $j \notin S$:
 - Calcular el [EDITAR] costo total de operar con $S \cup \{j\}$.
 - Seleccionar $j^* = \arg \min_j [\text{EDITAR}] \text{CostoTotal}(S \cup \{j\})$.
 - Añadir j^* a S .
 - Eliminar columna j^* de la matriz M mediante la actualización de costos variables teniendo en cuenta la nueva apertura.
 - Aplicar restricciones:
 - [EDITAR] Por cada restricción del enunciado (capacidad, cobertura, distancia, etc.):
 - ◇ [EDITAR] Construir matriz auxiliar con los parámetros relevantes.
 - ◇ [EDITAR] Verificar si la inclusión de j^* permite cubrir clientes pendientes o satisfacer condiciones.
 - ◇ [EDITAR] Actualizar estado de cobertura, demanda, penalizaciones u otros atributos si aplica.
 - Evaluar si se cumple alguna condición de parada:
 - [EDITAR] ¿Se superó la capacidad total? ¿Se alcanzó el número máximo de CDs?
 - ¿Se alcanzó un mínimo de costo total o no mejora suficiente?

Salida:

- Conjunto final de CDs S .
- Costo total asociado a la solución.
- [EDITAR] Reportes adicionales según restricciones (clientes cubiertos, capacidad utilizada, etc.).

8.2. Algoritmo DROP

Notación Algoritmo DROP

Parámetros en todos los casos

- i : Puntos de venta que se pueden convertir en centros de distribución: $i = 1, 2, \dots, n$.
- j : Puntos de venta cuya demanda se debe satisfacer $j = 1, 2, \dots, m, \quad m \geq n$
- M : Matriz de distancias, donde M_{ij} es la distancia entre el posible centro de distribución i y el cliente j .
- c_{ij} : Costo por unidad de distancia recorrida entre el posible centro de distribución i y el cliente j .

Restricciones:

- P : Conjunto de productos ofrecidos $p \in P$.
- q_i : Capacidad de producción o de procesamiento del posible centro de distribución i .
- d_{jp} : Demanda de cada producto p en cada punto de venta j . $d_j = \sum_p d_{jp}$
- k_p : Costo unitario de cada producto $p \in P$.

Receta: Caso con restricción de capacidad.

Inicialización:

1. Encontramos matriz de costos variables e identificamos costos fijos asociados a cada posible centro de distribución i . En la diagonal deben haber espacios vacíos porque si ponemos ceros, Excel explota.

Iteración 1:

2. Expresamos la matriz de costos variables, costos fijos, costos fijos mínimos y costos totales. Quedará algo como:

i/j	1	2	...	m	CF	Mín CF	Mín CV	CT
1		CV_{12}	...	CV_{1m}	CF_1	$\sum_i CF_i - CF_1$	8.1	Mín CF_1 + Mín CV_1
2	CV_{21}		...	CV_{2m}	CF_2	$\sum_i CF_i - CF_2$	8.1	Mín CF_2 + Mín CV_2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
n	CV_{n1}	CV_{n2}	...	CV_{nm}	CF_n	$\sum_i CF_i - CF_n$	8.1	Mín CF_n + Mín CV_n

$$\text{Mín CV} = \text{MIN}(\text{INDICE}(\text{MatrizCV}; ; i)) \quad (8.1)$$

3. Encontramos CT mínimo. Denotaremos el Mín CF asociado a este como Mín CF^* .

4. Creamos matriz auxiliar para evaluar restricciones de puntos abiertos, eliminando el punto de venta que dio mínimo anteriormente. Esta sí deberá tener ceros en la diagonal. Quedará algo como

i/j	1	2	...	m	Demanda acum	Capacidad
1	0	CV_{12}	...	CV_{1m}	Consideración 1	Capacidad 1
2	CV_{21}	0	...	CV_{2m}	Consideración 1	Capacidad 2
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$	$\vdots$
n	CV_{n1}	CV_{n2}	...	CV_{nm}	Consideración 1	Capacidad n
Míns	Eq. 8.2	Eq. 8.2	...	Eq. 8.2		
Atendido por	Eq. 8.3	Eq. 8.3	...	Eq. 8.3		
Demanda	d_1	d_2	...	d_m		

Consideración 1: Sumamos las demandas considerando los puntos que son atendidos por cada centro abierto i . Estos se muestran en las últimas filas de la matriz auxiliar.

$$\text{mín } i = \text{mín}_j \{CV_{ij}\}, \quad \forall i \quad (8.2)$$

$$\text{Atendido por} = \text{INDICE}(\text{PuntosAbiertos}; \text{COINCIDIR}(\text{Mín } i; \text{MatrizCV})) \quad (8.3)$$

Demás iteraciones:

5. Procedemos desde el paso 2 pero ahora actualizaremos la matriz de costos variables eliminando las filas asociadas al menor costo total de cada iteración. En las demás iteraciones usaremos esta nueva forma de calcular costos fijos mínimos. Esta será

$$\text{mín } CF = \text{mín } CF^* - CF_i \quad \forall i$$

Esta función significa que agarraremos el costo fijo mínimo del óptimo de la iteración pasada y le restamos el costo fijo de cada posible punto de venta actual.

6. Finalizamos el algoritmo cuando el costo total aumente, cuando se incumplan las restricciones de capacidad o cuando se llegue el número de centros de distribución que nos pidan en el enunciado.

Receta: Pseudocódigo DROP**Pseudocódigo – Algoritmo DROP (Parte 1/2)****Inicialización:**

- Cargar matriz de distancias o costos M entre todos los nodos.
- Inicializar conjunto de CDs con todos los nodos: $S = \{1, 2, \dots, n\}$.
- Inicializar variables asociadas a restricciones (ej: demanda cubierta, capacidad, etc.).
- Definir criterios de parada:
 - [EDITAR] Restricción de viabilidad del enunciado (por ejemplo, mínimo de cobertura, número mínimo de CDs, capacidad mínima, etc.).
 - Minimización del costo total.

Paso 1 – Evaluación Inicial:

- Calcular el [EDITAR] costo total de operar con todos los nodos como CDs.
- Para cada nodo $j \in S$:
 - Calcular el [EDITAR] costo total de operar con $S \setminus \{j\}$.
 - Evaluar factibilidad de eliminar j según restricciones del enunciado.
- Si existe al menos un nodo cuya eliminación reduce el costo total y mantiene la viabilidad, eliminar el que más reduce el costo.

Pseudocódigo – Algoritmo DROP (Parte 2/2)

Paso 2 – Iteraciones hasta convergencia:

- Mientras se pueda eliminar un CD sin violar ninguna restricción:
 - Para cada $j \in S$:
 - Calcular el [EDITAR] costo total de operar con $S \setminus \{j\}$.
 - Verificar si se cumplen todas las restricciones con esa configuración:
 - ◊ [EDITAR] Generar matrices auxiliares para calcular cobertura, capacidad, asignación de clientes, etc.
 - ◊ [EDITAR] Comprobar si eliminar j permite seguir cumpliendo con todas las restricciones.
 - Si existe una eliminación válida, aplicar la que más reduce el costo total.

Salida:

- Conjunto final de CDs S .
- Costo total asociado a la solución.
- [EDITAR] Reportes adicionales según restricciones (clientes cubiertos, capacidad utilizada, etc.).

8.3. Fórmulas útiles para las macros

Generalmente todo lo que tiene que ver con las macros está muy bien explicado en los manuales correspondientes. No obstante, hay formas de automatizar el pasar la columna de distancias/costos a una matriz sin tener que ir copiando y pegando dato por dato:

Proceso columna a matriz:

Digamos que queremos pasar datos de una sola columna a una matriz de dimensiones $n \times n$. Será útil usar:

=INDICE(ColumnaDeDatos;TRANSPONER(SECUENCIA($n;n;1;1$)))

Proceso matriz a columna:

Para implementar un proceso inverso, en donde tenemos una matriz de dimensiones $n \times n$ y queremos ponerlo en una columna, será útil usar:

=TRANSPONER(APIARH(Columna1;Columna2;...;Columna n)))